

OSDI '26

Prism: Cost-Efficient Multi-LLM Serving via GPU Memory Ballooning

Shan Yu[1], Yifan Qiao[2], Mingyuan Ma[3], Yangmin Li[4], Shuo Yang[2], Xinyuan Tong[5], Yang Wang[6], Zhiqiang Xie[7], Yuwei An[4], Shiyi Cao[2], Ke Bao[8], Deepak Vij[9], Xiaoning Ding[9], Yichen Wang[9], Qingda Lu[10], Zhong Wang[11], Gao Gao[12], Harry Xu[1]*, Junyi Shu[1]*, Jiarong Xing[13]*, Ying Sheng[1]*

* 通讯作者;

[1] UCLA
[2] UC Berkeley
[3] Harvard University
[4] CMU
[5] University of Edinburgh
[6] Intel
[7] Stanford University
[8] LMSYS
[9] ByteDance
[10] Alibaba Cloud
[11] Tsinghua University
[12] Novita AI
[13] Rice University

第一作者 / 通讯作者所在单位



[1] UCLA

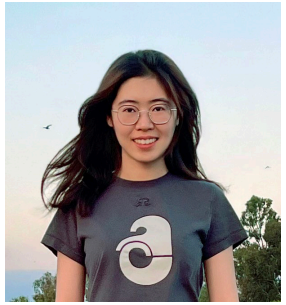


[13] Rice

汇报人：江建谕

2026.11.9

作者介绍-第一作者与研究团队



Shan Yu

论文第一作者 · UCLA 博士候选人

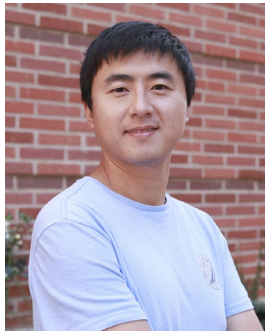
导师：Harry Xu

本科 / 硕士：浙江大学

研究：低成本、低延迟的复合 AI 工作负载系统

经历：曾任 Intel 高级 AI 框架工程师、BigDL 开发者

Amazon AI PhD Fellowship 2025



Harry Xu

通讯作者之一 · UCLA 计算机系教授

Shan Yu 的博士导师

研究：运行时 / 操作系统、分布式系统、AI 基础设施

Open Virtual GPU Project；kvcached 相关研究

ACM Distinguished Scientist · Dahl–Nygaard Junior Prize 2018

团队相关工作

- 在线 / 离线协同：ConServe: Fine-Grained GPU Harvesting for LLM Online and Offline Co-Serving (ICML’ 26)
- 视频分析：VQPy: An Object-Oriented Approach to Modern Video Analytics (MLSys’ 24)
- 分布式共享内存：DRust: Language-Guided Distributed Shared Memory with Fine Granularity, Full Transparency, and Ultra Efficiency (OSDI’24)

来源：web.cs.ucla.edu/~shanyu/；web.cs.ucla.edu/~harryxu/（论文角色按首页，师生关系按主页）

作者介绍-推理系统与开源生态

研究方向

- 大模型推理系统与服务调度；SGLang / S-LoRA
- 机器学习系统、形式化方法与自动推理

相关代表工作

- SGLang: Efficient Execution of Structured Language Model Programs (NeurIPS'24)
- S-LoRA: Serving Thousands of Concurrent LoRA Adapters (MLSys'24)
- Fairness in Serving Large Language Models (OSDI'24)

与本文关系：通讯作者之一；Prism 基于 SGLang 实现



Ying Sheng（盛颖）

RadixArk 联合创始人兼 CEO

Stanford 博士（2023）

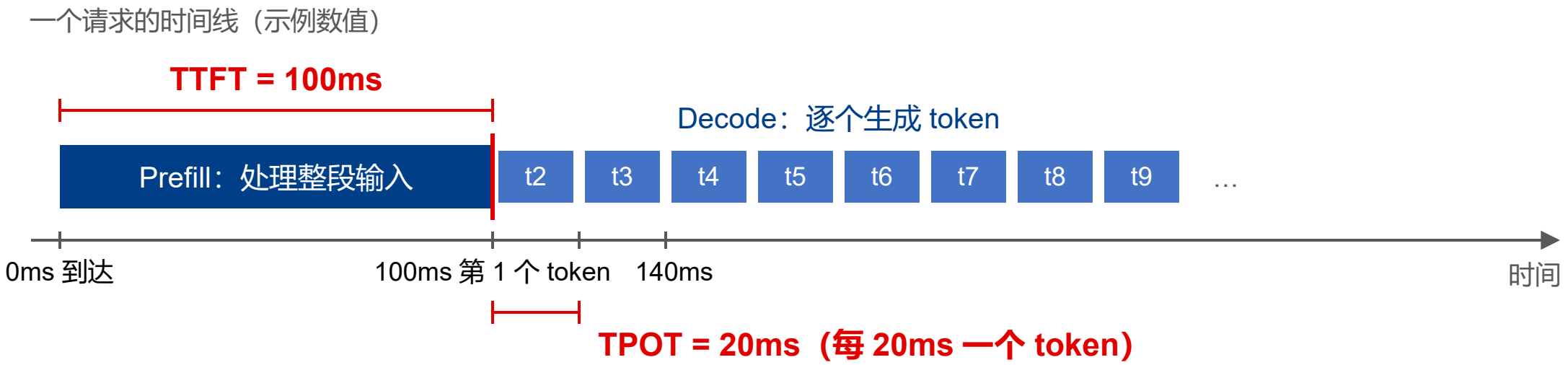
论文署名单位：UCLA

目录

- 研究背景
- 研究动机
- 相关工作
- 设计实现
- 实验评估
- 总结与思考

研究背景-多模型服务与 SLO

- **现状**：推理服务商要托管成千上万个基座与微调模型，很多流量低，但必须随时可用
- **独占 GPU 组**：响应最快，但 GPU 占空比通常 **< 30%**
- **TTFT（首 token 时延）**：请求到达 → 吐出第一个 token，决定“等待感”
- **TPOT（每输出 token 时延）**：相邻输出 token 的平均间隔，决定“流畅感”
- **SLO attainment**：满足 SLO (服务等级目标)的请求占比，TTFT、TPOT 分开统计
  - 例：100 个请求中 95 个 TTFT 达标 → TTFT attainment = 95%



研究背景-显存构成

- 大模型推理**访存密集**，显存主要被两样东西占据
- **模型权重**：大小固定、加载后一直占着
  - 例：70B 模型 FP16 约 **140GB**，一张 80GB 卡放不下
- **KV cache**：缓存每个 token 的 K、V
  - 动态：随序列长度和 batch 增长
  - 输出多长事先不知道
- **PagedAttention**（SGLang / vLLM）：
  - 预留一大块显存作 KV 块池，按块分配
  - L 层每层各一个 K、V 张量
  - 形状 (B, T, H, D)：块数、每块 token 数、KV 头数、头维度

$$\mathrm{KV/token} = 2 \times L \times H \times D \times 2\ \mathrm{B}$$

模型	(L, H, D)	每 token KV
Llama-3-8B	(32, 8, 128)	**128KB**
GPT-OSS-120B	(36, 8, 64)	**72KB**

(L, H, D) 取自论文 §2；每 token KV 按 FP16 由汇报人计算

KV 块大小因模型而异 → 无法简单地放进同一个张量池共用

研究背景-GPU 共享的两种方式

- 推理**访存密集**，尾延迟主要由显存决定 → 论文聚焦**显存共享**（算力共享已有 MIG、vGPU）

空间共享（MuxServe）

多个模型**同时常驻**
显存按模型切分

适合：几个低流量、但一直有请求的模型，无切换代价

缺点：显存边界基本静态，模型空闲了，它占的显存别人也用不上

时间共享（QLM、Aegaeon）

模型按需**换入换出**
空闲模型被踢出显存

适合：零星请求的模型，空闲时资源完全释放

缺点：换入要加载权重 + 初始化引擎，常需**数秒**；请求一密集就频繁切换

借鉴：VM memory ballooning

虚拟机里装一个 **balloon driver**

宿主机要回收内存时：气球在虚拟机内**膨胀**，占住暂时不用的内存交还宿主机

虚拟机需要时：气球**缩回**

→ 内存在虚拟机之间**弹性流动**

生产环境的负载，更适合空间共享还是时间共享?

研究动机-生产 trace：bursty group

Trace name	Service provider	# models	Time span
Hyperbolic	Hyperbolic [19]	24	4 months
Novita	Novita AI [3]	16	1 month
Arena-Battle	Chatbot Arena [9]	129	16 months
Arena-Chat	Chatbot Arena [9]	84	11 days

Table 1: Production trace summary.

Table 1：生产 trace 概览

4 份 trace：Hyperbolic、Novita AI、Chatbot Arena（Battle / Chat）

24–129 个模型，11 天–16 个月

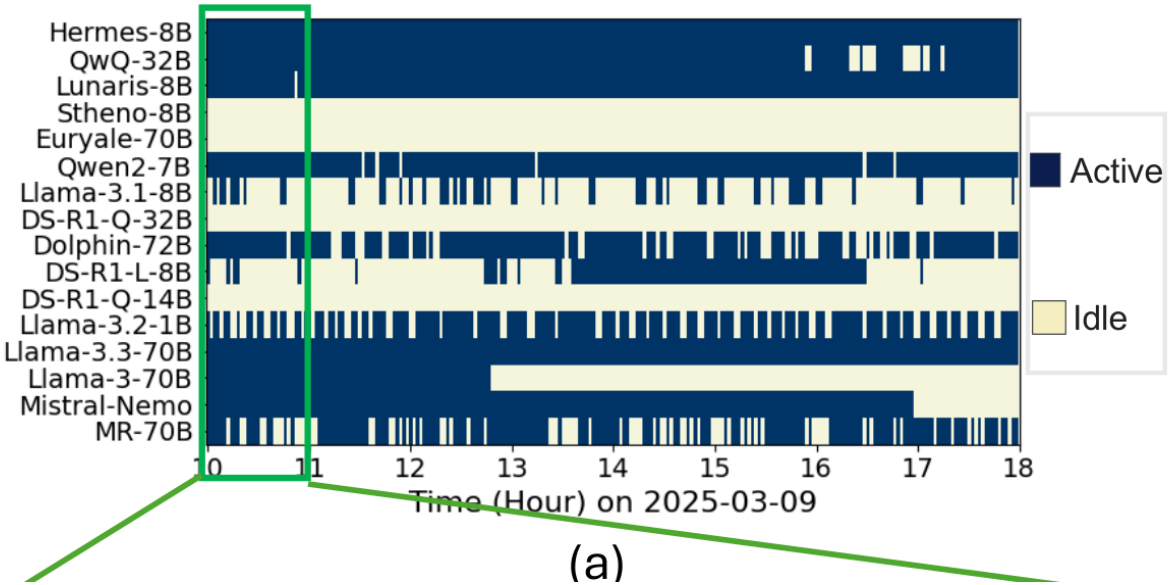
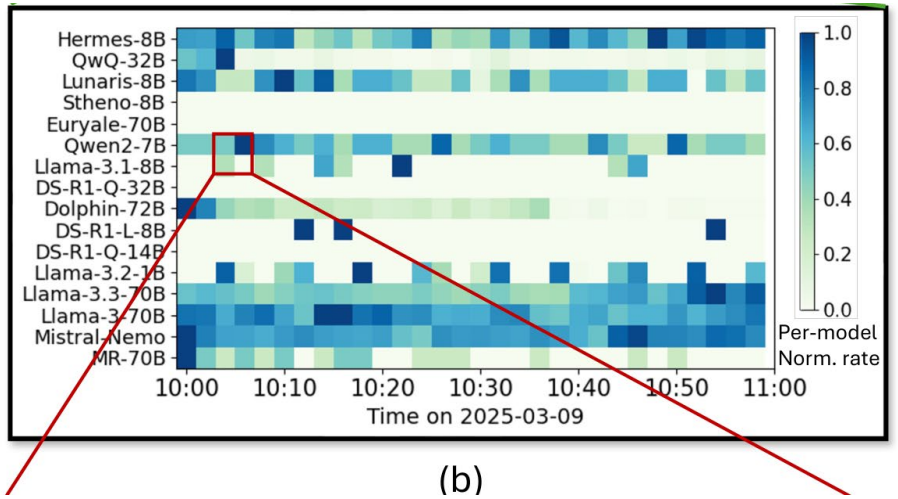


Figure 1(a)：Novita trace 上各模型的活跃情况

- **读图**：纵轴 16 个模型，横轴时间（10–18 点），每格 3 分钟；深蓝 = 活跃，浅黄 = 空闲
- **横看：Llama-3.3-70B 持续活跃；DS-R1-Q-14B 在本窗口几乎全为空闲**
- **竖看 bursty group**：**23%–50%** 的模型同时活跃，活跃集合每小时变 **54–766** 次

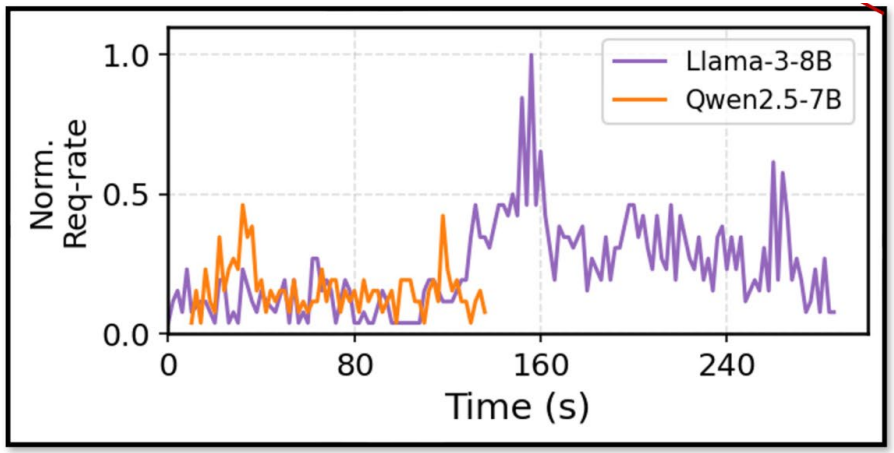
Novita 模型 70%+ 时间空闲 → 静态切分下 50% 显存被占着不用

研究动机-请求剧烈波动



(b)

Figure 1(b)：10:00–11:00 各模型请求率



(c)

Figure 1(c)：贯穿例子，(b) 红框内的 5 分钟

- **读 (b)**：颜色越深请求率越高（各模型自归一化）
  Qwen2-7B 在低谷与短暂高峰间快速振荡
- 很多模型 **CV > 1**；每小时 **40–100** 段 >10s 空闲
- 相邻两天 Pearson 相关 **≈ 0** → 无法按历史预测

CV = 标准差 / 均值，按每分钟请求数计算

① 交错阶段 0–130s

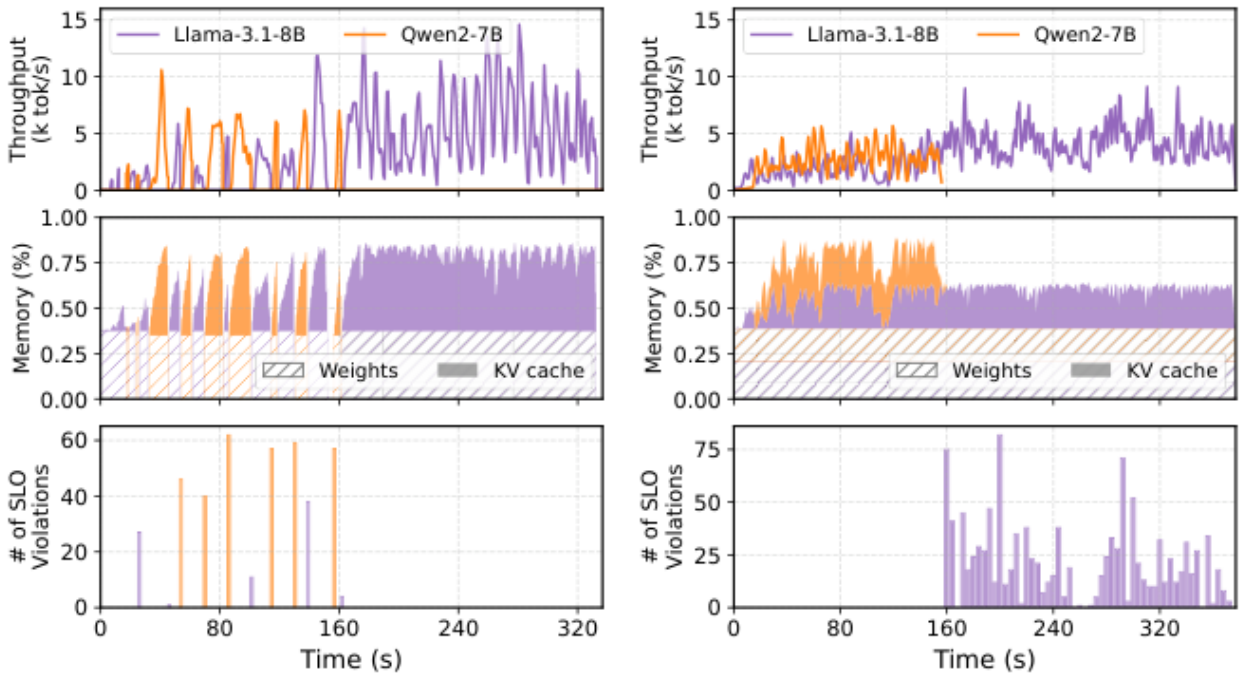
紫 Llama 8B 与橙 Qwen 7B
中等强度、交替出现
→ **同时争资源**

② 突发阶段 130s 后

Qwen 请求停止
Llama 突然爆发
(约 155s 达到峰值)

切换没有征兆 → 系统必须对瞬时变化做出反应

研究动机-现有方案为何失败



(a) Pure time sharing. (b) Pure space sharing.

Figure 2：用 Figure 1(c) 的 5 分钟 trace 跑两种方案
紫 Llama-3.1-8B，橙 Qwen2-7B

读图：左 (a) 纯时间共享，右 (b) 纯空间共享
三行：吞吐 / 显存（斜线权重、实心 KV）/ SLO 违约数

方案	交错阶段 0–130s	突发阶段 130s 后
(a) 时间共享 QLM	**反复换入换出，显存呈锯齿；Qwen 违约 40–62**	只剩 Llama，用满约 0.8 显存，违约 ≈ 0
(b) 空间共享静态切分	两个模型都常驻，违约 0	**Qwen 权重与分区仍锁着；Llama 卡在约 0.6，违约最高约 80**

交错阶段要空间共享，突发阶段要回收空闲显存 → 两种方案正好互补

研究动机-核心洞察

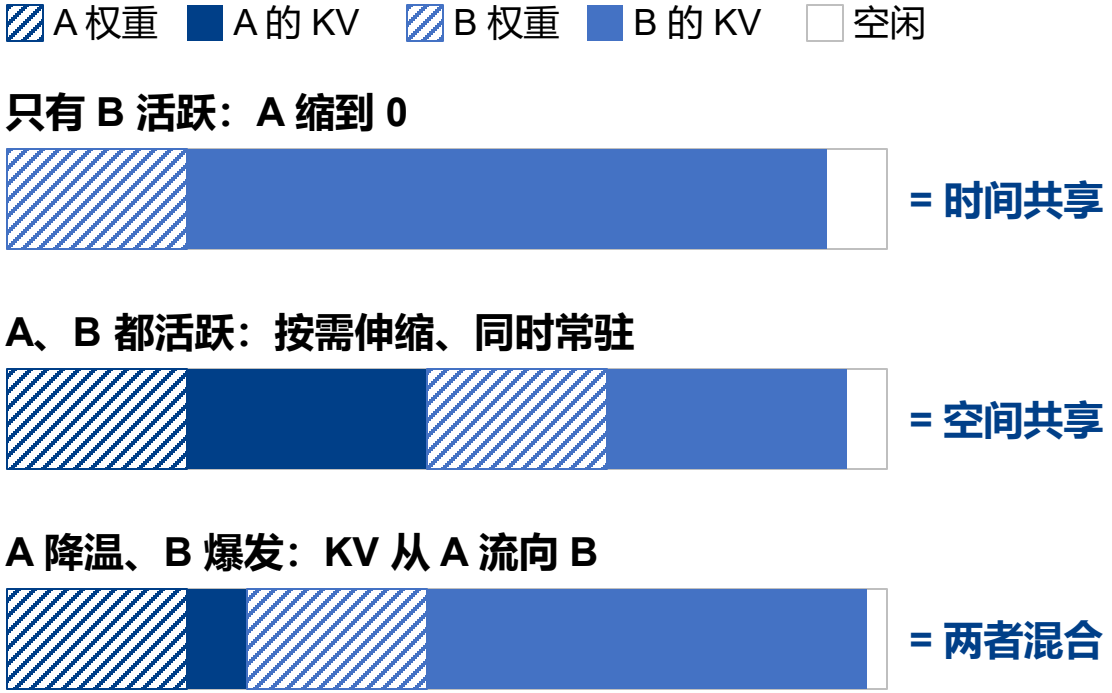
- **bursty group 需要时间共享能力**
  活跃集合在变 → 要能**踢掉空闲模型**
- **请求剧烈波动需要空间共享能力**
  请求交错 → 要让**活跃模型同时常驻**
- **简单切换两种策略会冲突**
  时间共享要抢占，空间共享要资源一直在

洞察：GPU 显存是统一的瓶颈

- 时间共享关心：高效换入**权重**
- 空间共享关心：并发模型间分配 **KV cache**

→ 把显存做成可 **“气球化”** 的弹性资源，统一两者

示意：一张 GPU 的显存随时间分配（非论文数据）



同一套机制，**不用切换策略**：只是边界在动

现有工作离这个目标还差多远?

相关工作

类别	代表工作	做法	在 Prism 看来的不足
空间共享	MuxServe	多模型同驻，按负载划分显存与算力	空闲模型钉在 GPU 上
时间共享	QLM、ServerlessLLM、Aegaeon	按需换入换出，快速冷启动	请求一交错就频繁切换
显存虚拟化	vAttention、vTensor	CUDA VMM 解耦虚拟地址与物理显存	只管单模型，需重写引擎
超卖调度	MSched	显存超卖后调度 GPU–CPU 换页	事后补救，不做事前避免
SLO 调度	Llumnix、DistServe	SLO 感知的请求调度	只针对单模型

组里讲过的相关论文

Punica：同基座多 LoRA 批处理 · **Weaver**：热门模型 attention 卸载到冷门模型的 GPU
HydraServe：多卡并行拉取降冷启动（近似 Prism 并行加载） · **Faro**：SLO 感知扩缩容（与 Prism 正交）

Prism：用弹性显存同时支持空间与时间共享，且对推理引擎透明

设计实现-总览

读图

- 请求 → Frontend → 各 **GPU group**
  一张卡或一组 TP 卡，是调度的最小单位
- 组内：共享请求队列 → 请求仲裁 → 模型
- 模型框之间的**虚线** = 显存边界可伸缩
- group n：Model 3 虚线框 **Time Share**（已换出），
  Model1/2 **Space Share**（常驻）→ 两种共享并存
- 两组都有 Model1，但颜色不同：
  组内编号，不是同一个模型

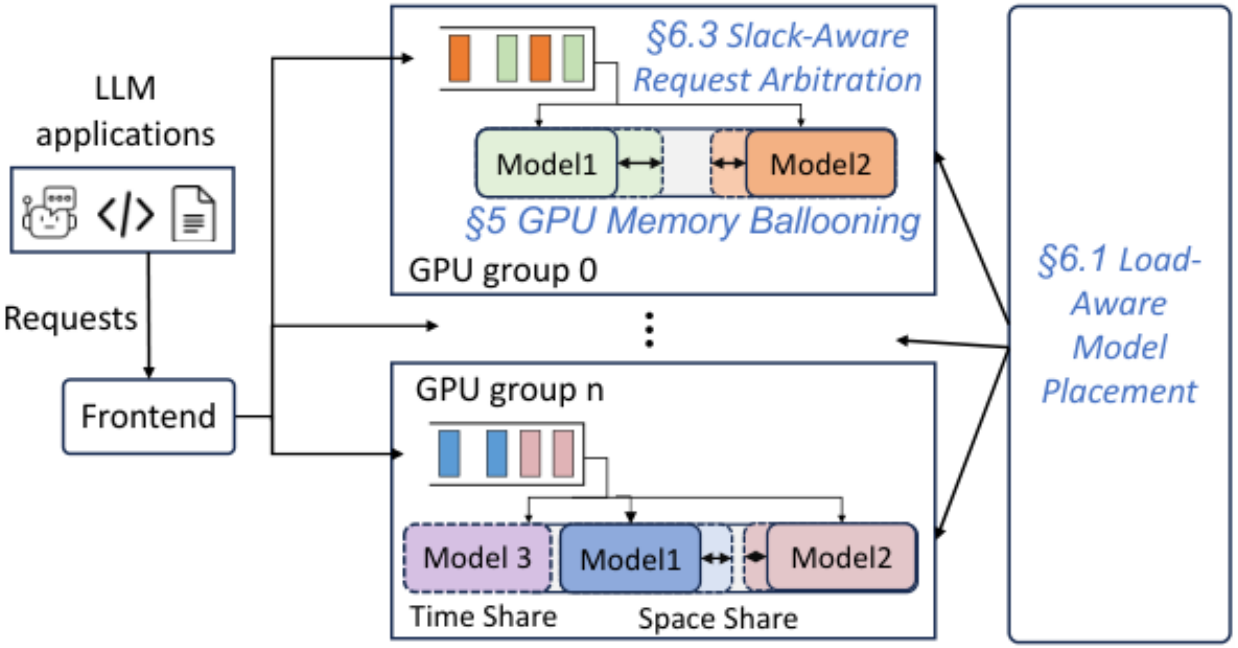


Figure 3：Prism 系统架构（图中 §6.3 即正文 §6.2）

数据面：kvcached 显存气球化（§5）

让显存在模型之间、权重与 KV 之间灵活流动

控制面：两级调度（§6）

全局模型放置（§6.1）：模型放哪张卡
GPU 本地请求仲裁（§6.2）：先服务哪些请求

数据面让显存能流动，控制面决定什么时候、往哪里流

设计实现-Prism 如何处理例子

Llama 权重　Llama 的 KV　Qwen 权重　Qwen 的 KV　空闲

示意，非论文实验数据（一张 GPU 的显存）

① 0–130s 交错



两个模型都常驻
权重各占一块
KV 按需伸缩：
谁请求多谁多拿

→ **像空间共享**

不用来回切换模型

② 约 135s Qwen 停



Qwen 手上请求完成
KV 页**立即回收**
交给爆发的 Llama
Qwen 权重**暂留**

以防它很快又来请求

③ 空闲约 45s 后



达到驱逐阈值
释放 Qwen 权重
Llama 几乎用满整卡

→ **像时间共享**

④ Qwen 再来请求



从**引擎池**取引擎
+ **并行加载**权重
8B 约 **0.7s** 重新激活

Llama 让出一部分 KV

对比 Figure 2：交错阶段避免了时间共享的**反复换入**，突发阶段避免了空间共享的**显存锁死**

显存怎样伸缩（kvcached）？什么时候、在哪里伸缩（控制面）？

设计实现-kvcached 的需求

- **问题**：SGLang / vLLM 启动即预占一大块物理显存作 KV 块池，没请求也占着
- 池子内部按块分配，但池子**不能缩、不能让**

四个需求

R1：权重与 KV 之间快速腾挪

换出模型的权重显存，马上给别人当 KV

R2：跨不同架构共享 KV

每 token KV 大小、层数各不相同

R3：开销小、碎片少

重分配很频繁，必须足够快

R4：对现有框架透明

KV 池在框架看来仍是一个大张量，不改 attention kernel

CUDA VMM：预留与映射分开（示意）

① 预留一段连续虚拟地址（不占物理显存）



物理显存：深色 = 本模型的页，浅色 = 别的模型可用

③ 不用就解映射，物理页还回去

程序看到的地址**始终连续**

背后的物理页**随时增减**

→ 这就是“气球”能伸缩的基础

引擎启动就占死物理显存；CUDA VMM 把预留和映射分开

设计实现-kvcached 的设计

- **核心思路**：显存管理下沉到 GPU 运行时层
  夹在引擎与显存之间，统一管理权重 + KV
  每引擎预留大段虚拟地址，物理页按需映射
- **D1 统一管理**：权重页释放后
  直接给别的模型做 KV；动态调每模型上限，
  有模型激活时收紧同卡其他模型
- **D2 按模型分页**：每个物理页只属一个模型
  看下层：每页只有一种颜色 → 不同 KV 大小不错位
- **D4 eTensor**：PyTorch 扩展，用法同普通张量
  不改 attention kernel，兼容 CUDA graph
  接入 SGLang 只改 **22 行**

D3（开销优化）见下一页

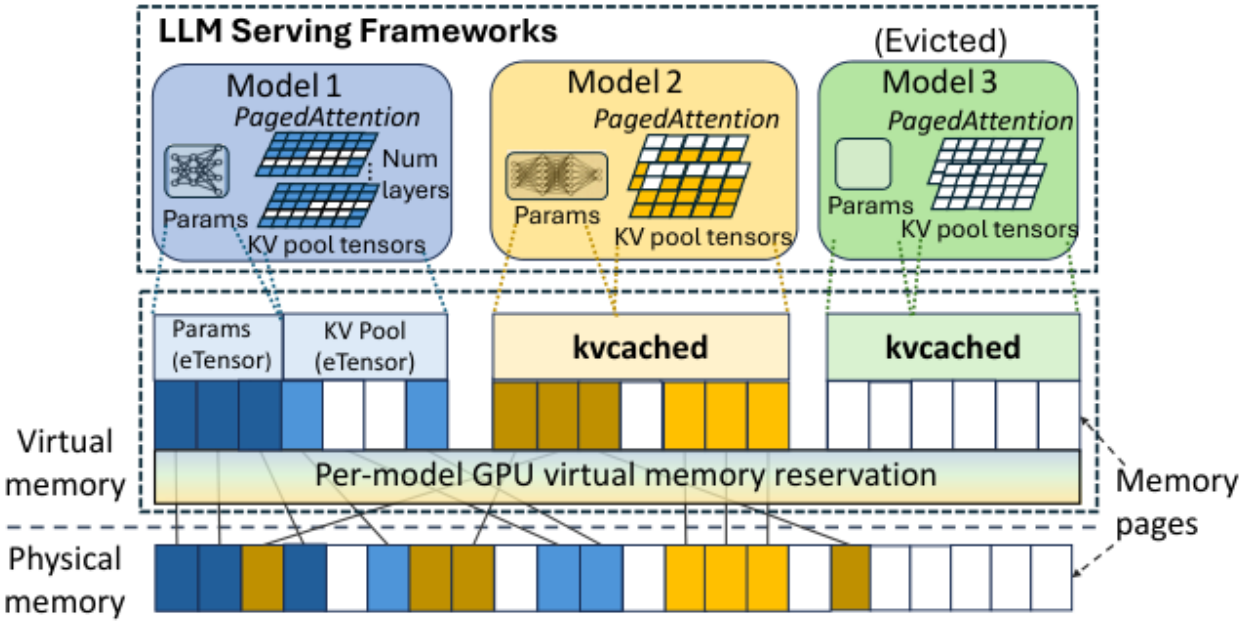


Figure 4：kvcached 的显存布局

读图：蓝 Model 1、黄 Model 2、绿 Model 3
上：框架视角 · 中：虚拟地址 · 下：物理页（交错）
深色格 = 权重页，浅色格 = KV 页，白格 = 未映射
Model 3 已驱逐：全是白格，但**预留还在**

引擎以为自己独占显存，物理页实际在模型之间流动

设计实现-kvcached 的开销优化

D3：降低重分配开销和碎片

- **所有层 K/V 连续存放**

  原来：每层各一个 K、V 张量，扩容要在 2L 个张量上分别映射

  现在：一个 token 所有层的 K/V 放在连续虚拟地址 → 一次批量映射（**2L× 加速**）

- **预分配**：后台线程备一小批物理页放 buffer

  申请、释放先走 buffer，不在关键路径上

- **2MB 页 + 优先填满半空页**

  页内按模型的 token 大小切分（所以 D2 要按模型分页）

  分配时先往半空的页里放 → 减少碎片

模型	每 token KV	每 2MB 页	每次扩容映射
Llama-3-8B	128KB	16 token（正好放满）	**64 次 → 1 次**
GPT-OSS-120B	72KB	28 token（剩 32KB）	**72 次 → 1 次**

映射次数 = 2L：Llama-3-8B L = 32，GPT-OSS-120B L = 36

每 2MB 页 token 数 = 2048KB ÷ 每 token KV，向下取整

以上数值由汇报人按论文 §2 的 (L, H, D) 计算

所有层 K/V 连续存放 → 一次映射代替 2L 次

设计实现-快速模型加载

- **激活两步**：① 初始化引擎 + 预留 KV 虚拟地址
  ② 从 CPU 内存加载权重；朴素做法可达**几十秒**

可复用引擎池（橙色）

引擎与模型生命周期解耦：驱逐只释放物理显存
引擎连同虚拟地址空间留在池里复用
KV 虚拟内存管理器按新模型重新对齐布局

并行加载（绿色）

多线程 cudaMemcpyAsync 到同一 GPU 会被串行
→ 权重切块，经多卡 PCIe 并行加载，再经 NVLink 汇聚
每卡只需约 30MB 流式 buffer，不干扰现有负载

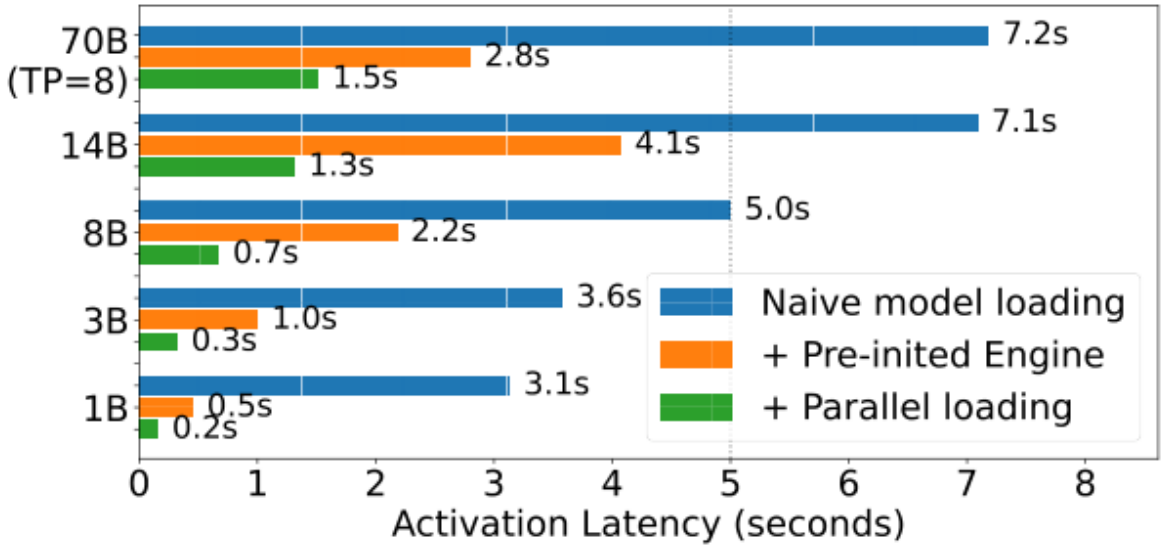


Figure 10：不同尺寸模型的激活时延

- 1B **3.1s** → **0.2s** · 8B **5.0s** → **0.7s** · 70B **7.2s** → **1.5s**
- 引擎池各尺寸约省 **3s**（基本是常数）
- 剩余部分随权重增长，并行加载再压 **2.5–3 倍**

0.2–1.5s 仍比 TTFT SLO（0.04–0.13s）高一个数量级

→ 激活要尽量少发生，需要控制面决定放哪、何时驱逐

设计实现-控制面的挑战与洞察

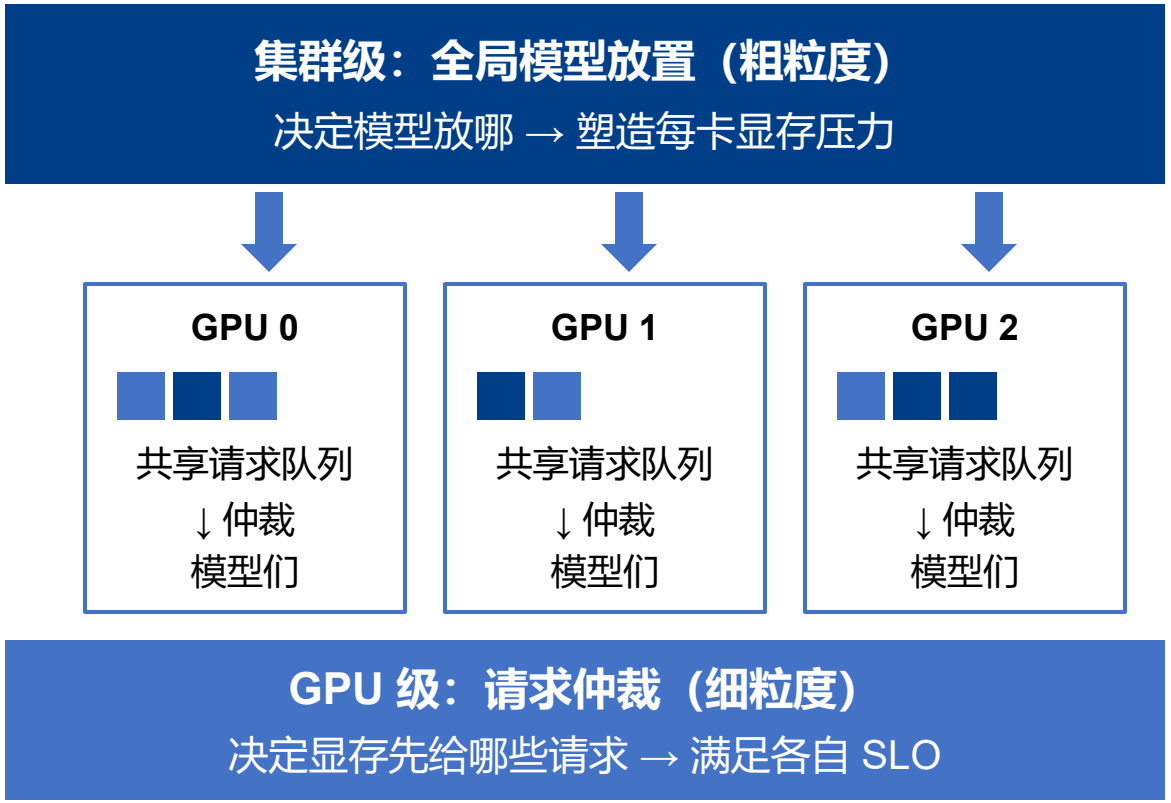
- **联合优化（ILP）不可行**
  - 放置：M 个模型放 N 张卡，**N^M** 种
  - 请求执行顺序：阶乘级组合
  - 放置与调度耦合：迁移会改变谁能服务哪些请求
  - 必须**在线**决策：看不到未来突发、输出长度、并发

洞察：从显存视角看，问题简化

模型、请求、token 的影响都体现在**显存争用**上

SLO 是否达标取决于执行时有无显存可用

→ 统一目标：**控制显存争用**，分两个尺度调度



- 仲裁优先保 **TTFT**：prompt 长度已知，prefill 时间可以估计

以“控制显存争用”为统一目标，分两级调度

设计实现-KV 压力比 KVPR

目标：每卡留足余量给活跃模型“吹气球”，避免同一突发组挤在一张卡上（输出长度未知 → 用启发式）

$$\mathrm{KVPR} = \frac{w_token_rate}{shared_kv} \qquad w_token_rate = \frac{token_rate \times token_size}{SLO}$$

- **shared_kv**：这张卡还能给 KV 用的显存 = 显存总量 − 所有模型权重
- **token_rate**：每秒新增 token（新请求输入 + 正在解码的输出），**60s** 滑动窗口平均
- **token_size**：每 token KV 大小（Llama-3-8B 128KB）；两者相乘 = 每秒新增 KV 显存
- **SLO**：用 TPOT SLO（解码占大头、对 KV 争用最敏感）；SLO 越严，权重越高
- 分子是该卡上**所有模型**的 w_token_rate 之和

→ **KVPR = 单位剩余显存要承受多急的显存需求，越高越难吹气球**

算例（示意，非论文实验数据） 数值由汇报人构造，w 为抽象单位

两张 80GB GPU；三个模型权重各 16GB；SLO 加权需求 w：**A = 6、B = 4、C = 2**

A 单独在 GPU0：6 ÷ (80 − 16) ≈ **0.094**　　B + C 在 GPU1：(4 + 2) ÷ (80 − 32) = **0.125**

下一页：贪心算法怎样得到这个放置?

设计实现-全局放置算法

Algorithm 1：在线贪心重放置

① **排序**：按 t·tz/s 降序，需求大的先放

② **逐个放置**：放到 KVPR 最小的 GPU
压力最小的卡常放着低需求模型 → 高低需求互补

③ **防无谓迁移**：当前卡 KVPR − 最佳卡 KVPR > τ
在线重算时比较新方案与原位置；小收益不搬动

④ **更新：需求 + t·tz/s，KV 容量 − 模型权重**
t = token_rate，tz = token_size，s = SLO

步骤	GPU0 KVPR	GPU1 KVPR	决策
放 A（w=6）	0	0	→ GPU0：6/64 ≈ 0.094
放 B（w=4）	0.094	**0**	→ GPU1：4/64 ≈ 0.063
放 C（w=2）	0.094	**0.063**	→ GPU1：6/48 = 0.125
结果	A 独占	B、C 共享	**高低需求互补**

沿用上页算例（示意，非论文实验数据）；红色 = 当前最小 KVPR

不仅用于初次放置：token rate 变化就可能打破原有均衡，不必先驱逐模型。

每轮从零初始化的是“规划统计”，不是先卸载模型；当前设备 g 是上轮位置。

best 与 current 的压力差 > τ 才迁移；否则保持原位置。重算周期 / τ 数值未披露。

贪心选择 ≠ 全局最优；附录给出压力上界。TP 分片仍使用反亲和约束。

迁移是对运行中负载变化的再平衡；阈值用于抑制频繁小幅调整

设计实现-驱逐、激活与迁移

驱逐

触发：空闲超过阈值
且同卡其他模型需要资源

动作：换到 CPU 内存

阈值：经验值，按空闲间隔
分布定，实验约 **45s** 最好

例子：Qwen 停止请求
约 45s 后释放权重

激活

触发：被驱逐的模型
来了新请求

位置：KVPR 最低的 GPU

动作：引擎池取引擎
+ 并行加载权重

即第 14 页的快速加载
0.2–1.5s

迁移

触发：放置算法要挪卡

不停服：源实例继续服务，
目标就绪再切换

路径（按可用性）：
① NVLink（权重 + KV）
② GPUDirect RDMA
③ 驱逐 + 重新激活
（<70B 最多几秒）

- **实测**：2 张 GPU 跑 8 个模型，10 分钟内只有 **2 次激活、3 次迁移**；8B 经 NVLink 迁移约 **20ms**

放置调整并不频繁，代价也小

设计实现-请求仲裁

- **问题**：各引擎独立队列，显存一空就贪心接请求

  高请求率、宽 SLO 的模型占满显存

  → **饿死**严格 SLO 的模型

- 每模型设显存上限？很难设

  保守 → 限吞吐；宽松 → 饿死别人

  调整要等请求完成才释放，可达数秒

做法：每卡一个共享请求队列

统一仲裁谁能用这张卡的显存

依据 **slack**：截止时间与所需执行时间的余量

- **移除：只从本轮优先集合 S 中移走**

  不是已经执行，也不能据此断言丢弃请求

$$d_r = a_r + s_r, \quad e_r = \frac{p_r}{c_r}$$

a 到达时间，s TTFT SLO → d 截止时间

p prompt 长度，c 该模型 chunked prefill 速度 → e 执行时间

Algorithm 2：Moore-Hodgson

① 所有请求按截止时间 d 升序排序

② 依次加入调度列表 S，累加执行时间

③ 刚加入的赶不上截止 → 从 S 中移除 **e 最大**的请求

④ **按 S 中保留的截止时间顺序派发**

设计实现-请求仲裁算例

请求	到达 a	执行 e	截止 d
r1	0	4	4
r2	0	2	5
r3	0	2	6

示意，非论文实验数据（参数由汇报人构造）

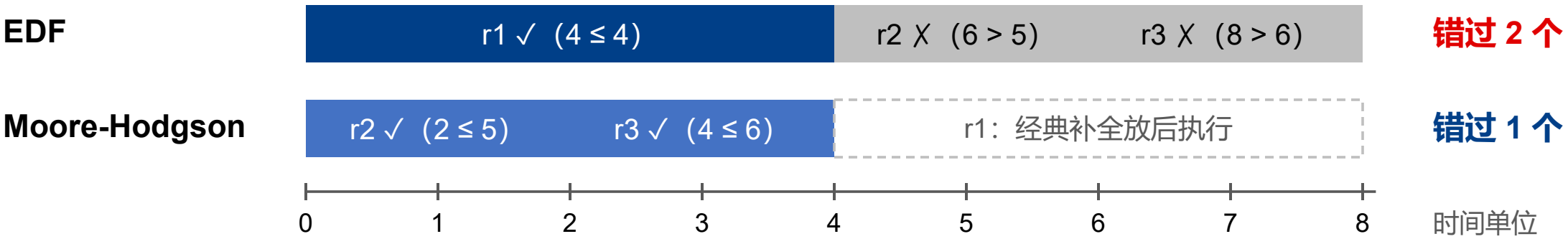
Moore-Hodgson 逐步（按 d 排序：r1 → r2 → r3）

① 加入 r1：累计 4 ≤ 4 ✓ 保留

② 加入 r2：累计 6 > 5 ✗ → 移除 S 中 e 最大的 **r1**，回到 2

③ 加入 r3：累计 4 ≤ 6 ✓ 保留

→ S = [r2, r3]；r1 让出按时优先资格



- **r1 本可按时完成；这里主动让出优先资格，以多保住一个请求，而非丢弃 r1。**
  - 虚线 r1 为经典算法的完整排程示意；Prism 论文只返回 S，未说明未选请求兜底策略。

放弃一个长请求，换来两个短请求达标

实验评估-实验设置

- **实现**：约 10,400 行 Python + 774 行 C++，后端 SGLang（接入改 22 行）
  Redis 前端队列；全局调度器独立进程，经 ZeroMQ 与引擎通信
  TP 模型的本地调度只在 rank 0 运行，再广播给其他 rank
- **硬件**：4 台 × 8 张 H100-80G，600GB/s NVLink，机间 100Gbps 以太网
- **Trace**：两份真实 trace
  Hyperbolic（推理服务商）：突发更强、负载更重
  Arena-Chat（Chatbot Arena）：模型更多、单模型请求率更低
  请求数乘以 **rate scale** 放大负载，保持原流量模式
- **模型**：58 个（见右表）；大规模实验用全部，
  其他实验按需选子集

Model size	1B-3B	4B-8B	9B-30B	31B-70B
#LLMs	43	8	3	4

Table 3: Models used in our evaluation.

Table 3：实验用模型（共 58 个）

指标：TTFT、TPOT 的 SLO attainment

SLO = 每个模型独占 GPU 时的 P95 延迟（TTFT **0.04–0.13s**，TPOT **5.2–50.9ms**）× 放宽系数 **SLO scale**

4 × 8 H100、两份真实 trace、58 个模型，看 TTFT / TPOT 达标率

实验评估-对比基线

- **S-Partition**（静态切分）：每模型固定一块
  显存常驻，纯空间共享（如 MIG、K8s 分数 GPU）
- **MuxServe++**：原版基于 vLLM、只支持 Llama-2
  作者移植到 SGLang，并用 kvcached 支持异构模型
  右表：三个 Llama-3.1-8B，10 分钟
  平均 TTFT **1.47s → 0.089s**
  平均 E2E **7.40s → 5.25s**，吞吐基本相同
- **QLM**：时间共享，按请求组派发、组间换模型
- **ServerlessLLM**：serverless 式时间共享
  空闲卸载，请求到来时在启动最快的服务器上激活

	Mean E2E (s)	P95 E2E (s)	Req Tput (r/s)	Token Tput (t/s)
MuxServe	7.40	18.85	7.97	3363.53
MuxServe++	5.25	12.09	7.98	3353.09
	Mean TTFT (s)	P95 TTFT (s)	Mean TPOT (ms)	P95 TPOT (ms)
MuxServe	1.47	11.04	21.21	31.95
MuxServe++	0.089	0.320	18.82	33.97

Table 2: Performance comparison of MuxServe and MuxServe++

Table 2：MuxServe 与 MuxServe++ 对比

不比通用 GPU 共享系统：它们面向单卡、与应用无关的负载，不针对 LLM 的 TTFT / TPOT

作者加强了基线，没有拿弱版本来比

实验评估-端到端：请求率扩展

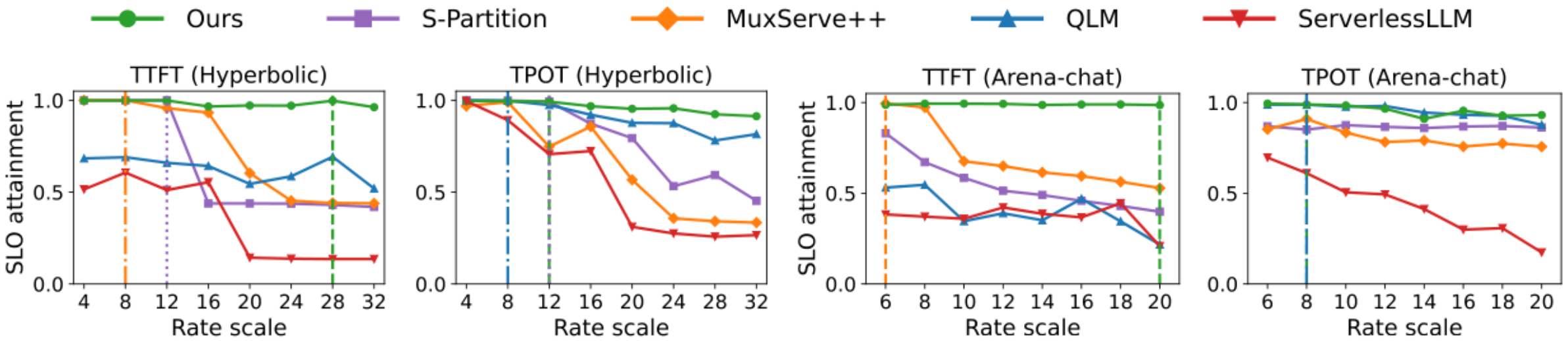


Figure 5 第 1 行：2 张 GPU 跑 8 个模型，横轴请求率放大倍数

验证：固定 GPU 数时，系统能承受多大的请求负载?

方法：2 张 GPU、8 个模型；回放两份 trace，逐步放大请求率，与四个基线比较。

读图：横轴负载、纵轴达标率；曲线右上更好。竖线为论文标注的 99% 位置。

结果：Arena-Chat 在 20× 负载时，Prism 的 TTFT 达标率仍约 99%，MuxServe++ 约 53%。

同样的 GPU 资源下，Prism 更能承受负载增长

实验评估-端到端：SLO 放宽

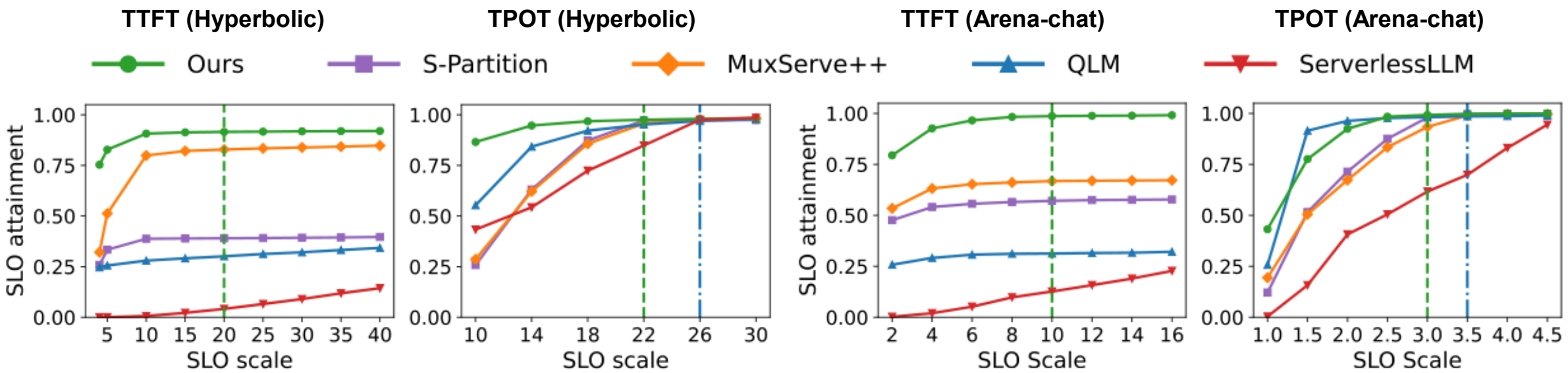


Figure 5 第 2 行：负载固定，横轴 SLO 放宽倍数（列标题为汇报人补注）

验证：改变延迟要求后，各系统的达标率如何变化?

方法：固定负载与 GPU 配置，只调整模型基准 SLO 的放宽倍数。

读图：横轴 SLO 放宽倍数、纵轴达标率；左上更好，即严格 SLO 下也能达标。

结果：Arena-Chat 上 Prism 在约 10× 时 TTFT 达标率约 99%；MuxServe++ 最高约 67%。

Prism 在更严格的 SLO 下仍保持较高达标率

实验评估-端到端：GPU 数量

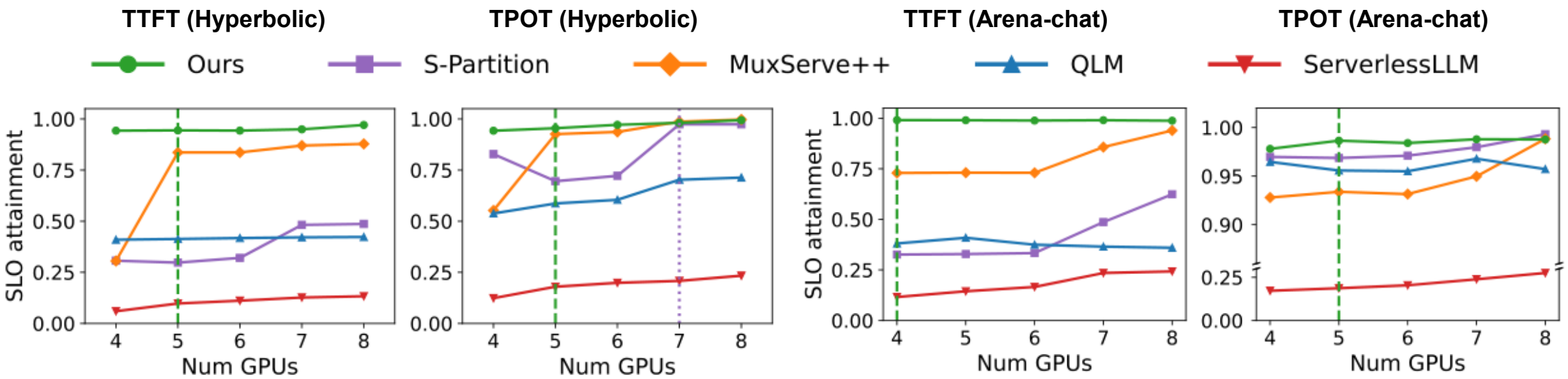


Figure 5 第 3 行：18 个 1–8B 模型（均可单卡放下），横轴可用 GPU 数

验证：保持服务质量需要多少张 GPU?

方法：18 个 1–8B 模型，逐步把可用 GPU 从 4 张增至 8 张，回放两份 trace。

读图：横轴 GPU 数、纵轴达标率；左上更好。右下 TPOT 图有断轴。

结果：Arena-Chat 的 TTFT：Prism 用 4 张卡约 99%，MuxServe++ 用 8 张卡约 94%。

在相近服务质量下，Prism 可以使用更少 GPU

实验评估-跨模型显存共享

验证：空闲模型的 KV 容量能否借给热点模型?

方法：Arena-Chat 的两模型 trace，

对比按需共享与每模型静态分区。

读图：三行共用时间轴，依次是请求率、

总 KV 用量、总吞吐；

结果：切换后 Prism 的 KV 用量接近 0.95，

静态分区约 0.6，吞吐也更高（归一化值）。

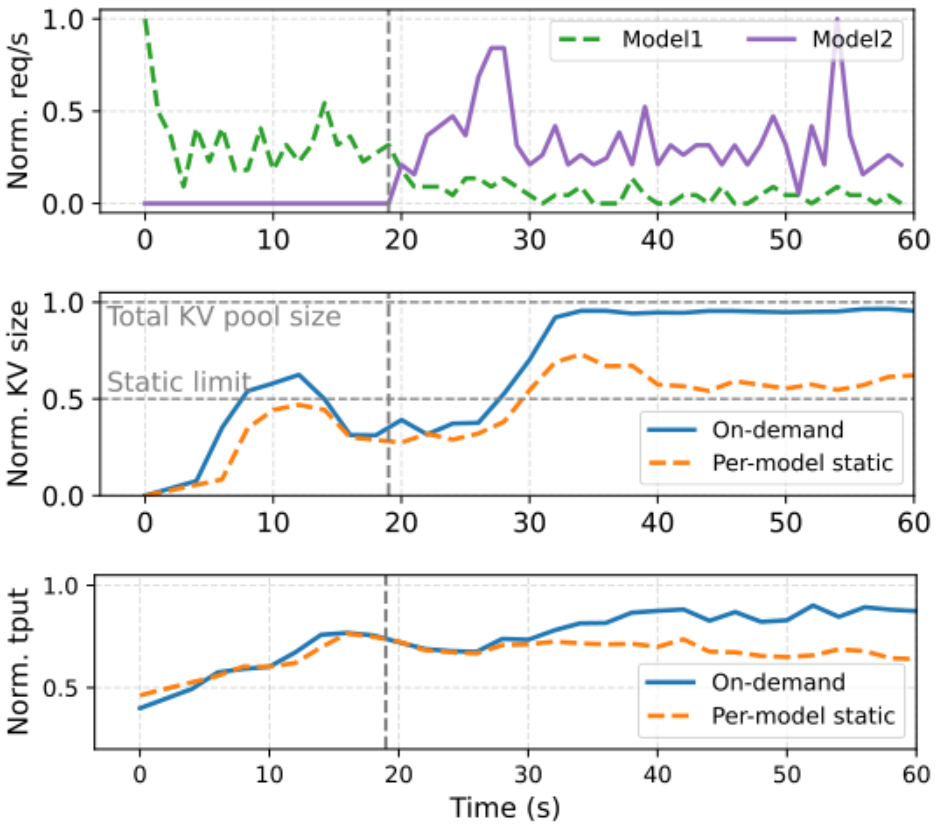


Figure 6：跨模型显存协调

实验直接验证：低需求模型的显存可以被热点模型利用

实验评估-全局放置

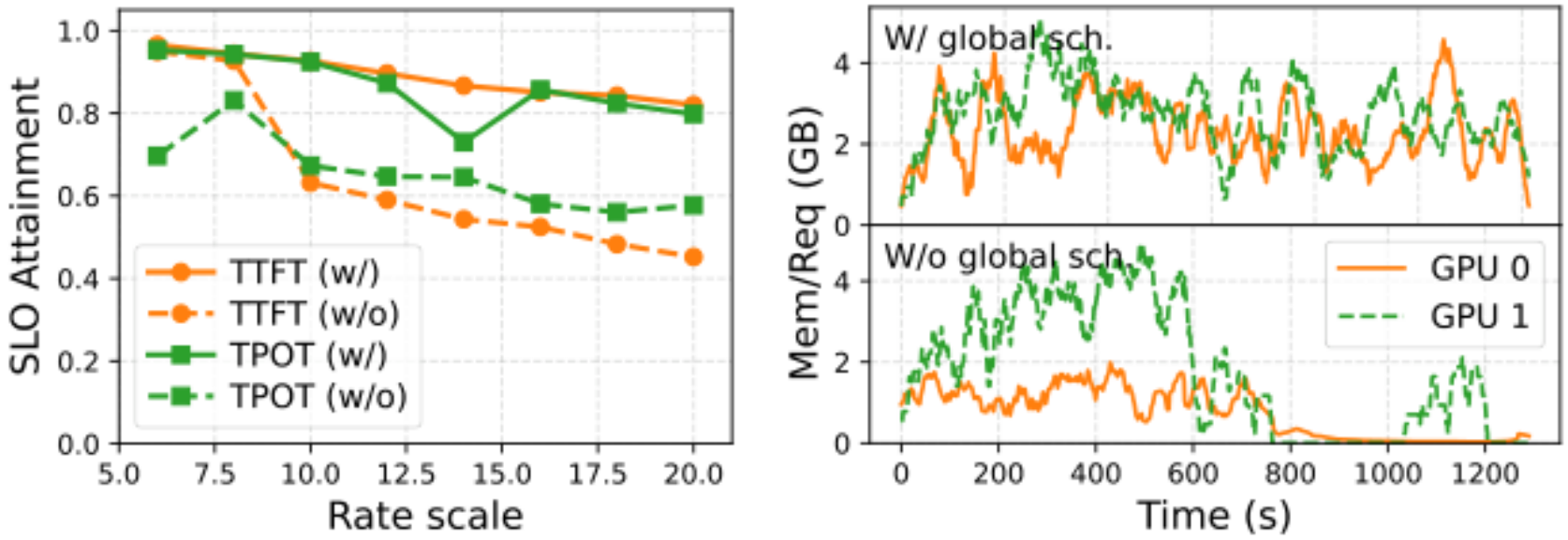


Figure 7：全局放置的作用（2 张 GPU、8 个 Arena-Chat 模型，开 / 关全局调度器）

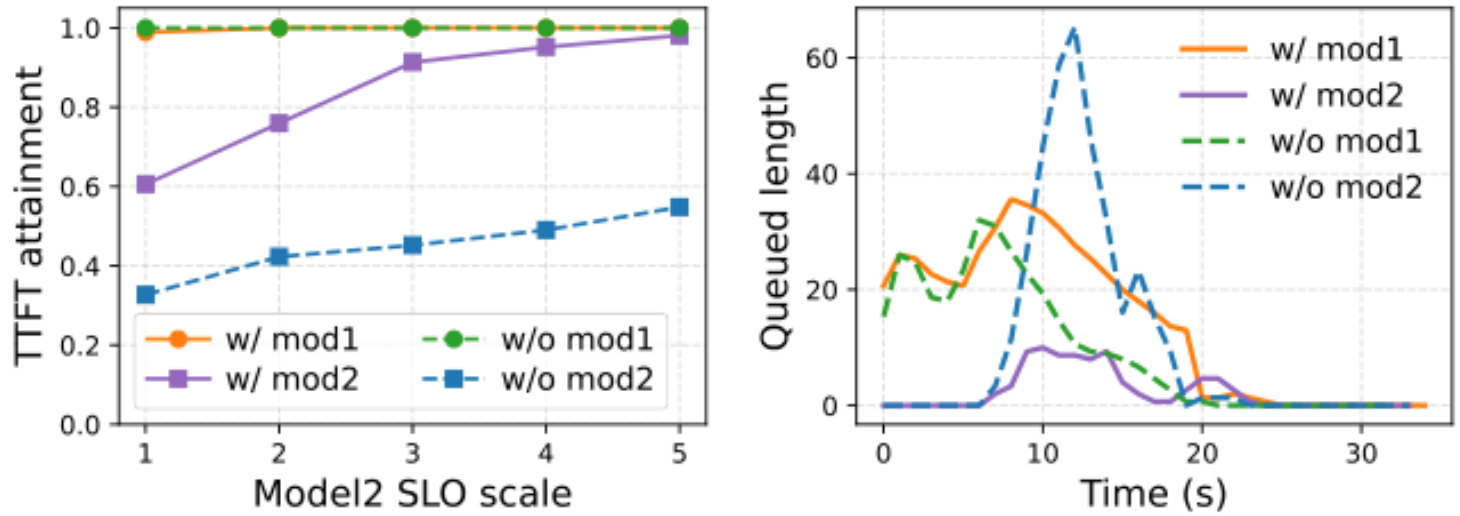
验证 / 方法：2 张 GPU 服务 8 个 Arena-Chat 模型，只比较开启与关闭全局放置。

读图：(a) 达标率越高越好；(b) 看两张卡每请求可用 KV 是否均衡，不比较时间轴左右。

结果：20× 负载时 TTFT 达标率约 0.82 vs 0.45；全局调度缓解一卡过载、另一卡空闲。

全局放置负责把显存压力在 GPU 之间摊平

实验评估-请求仲裁



(a) Attainment with SLO scales (b) Queue length with time

Figure 8：请求仲裁的作用（Model1 SLO 固定 8×，只放宽 Model2）

验证 / 方法：两模型同卡；Model1 的 SLO 固定为 8×，调整 Model2 的 SLO，比较有无仲裁。

读图：(a) 相同 SLO 下达标率越高越好；(b) 看队列积压，不是所有请求都越早完成越好。

结果：Model2 的 SLO 为 3× 时，TTFT 达标率约 91% vs 45%；Model1 仍接近 99%。

局部仲裁让宽松请求适度等待，优先保住紧 SLO 请求

实验评估-大规模部署

验证：扩大模型和集群规模后，是否仍能省卡?

方法：58 个模型、最多 32 张 H100；

32B 用 TP=4，70B 用 TP=4 或 8。

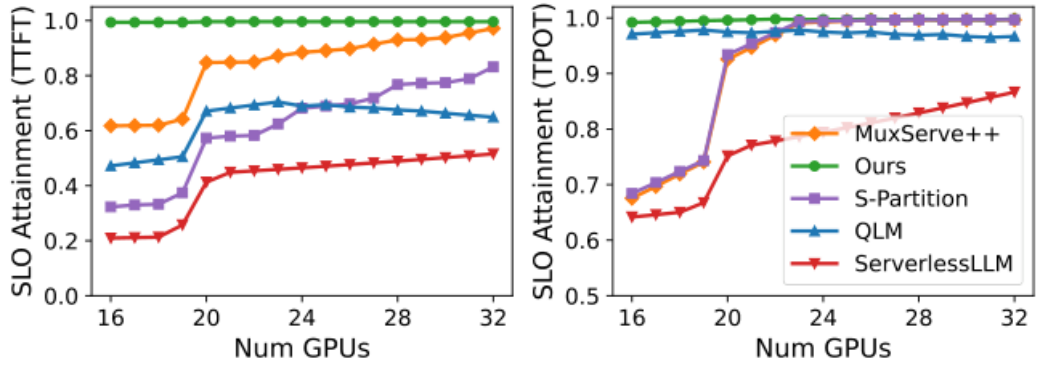
读图：(a) 达标率曲线左上更好；

(b) 99% 达标所需 GPU 越少越好，

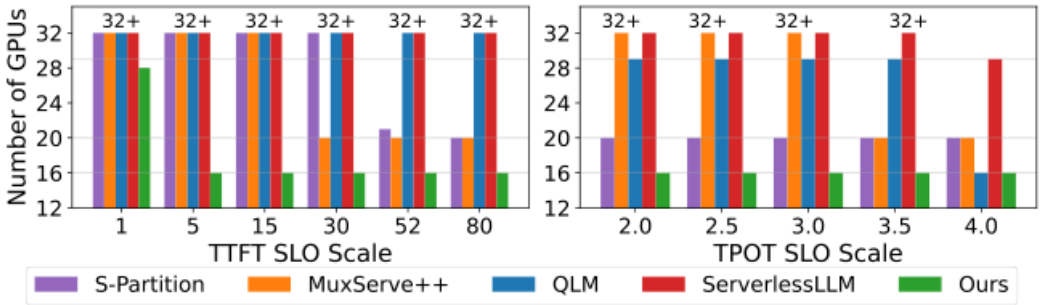
“32+”表示需要超过32张卡。

结果：(a) Prism 16 张卡的 TTFT 接近 99%，

MuxServe++ 32 张卡接近 97%。



(a) SLO attainment with cluster size scaling



(b) Number of GPUs needed for 99% SLO attainments

Figure 9：大规模部署

扩展到 58 个模型后，Prism 仍能降低 GPU 资源需求

实验评估-系统开销

方法：1 张 A100-40G、两个 Llama-3.2-3B；恒定请求率、无空闲，对比静态分区。
放置和请求顺序不变，主要隔离 kvcached 动态映射 / 解映射带来的开销。

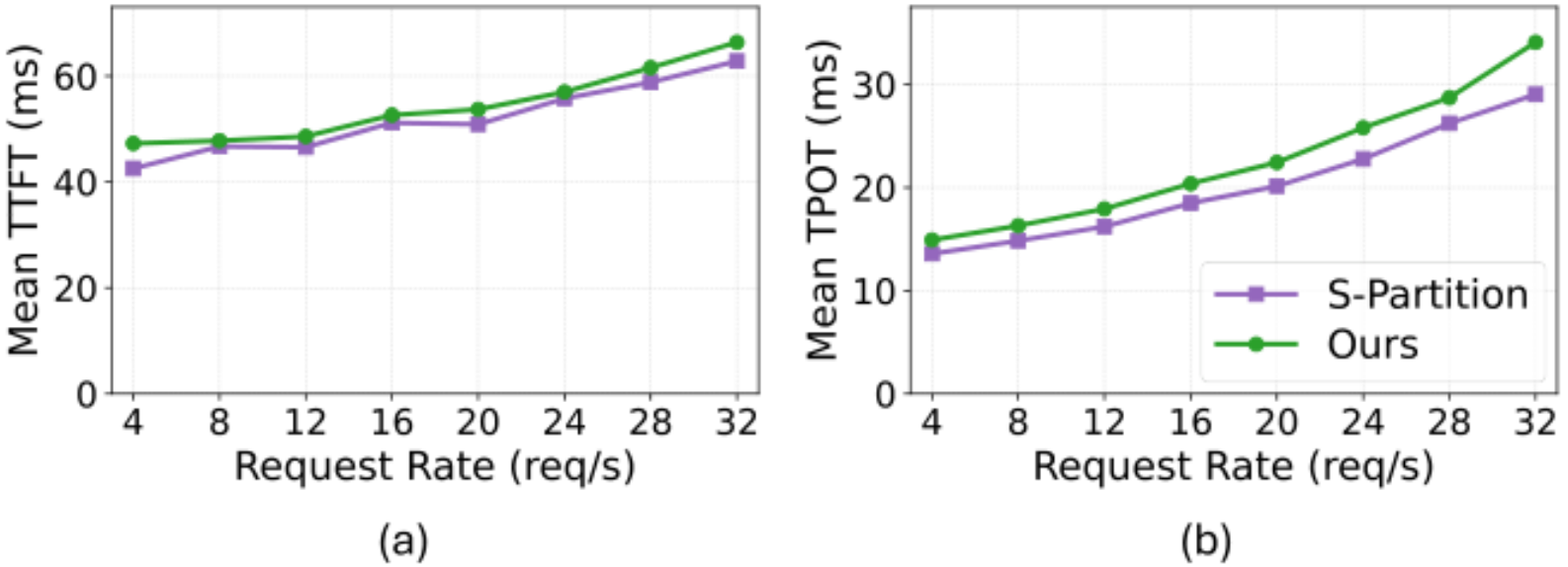


Figure 14：系统开销（紫 S-Partition，绿 Prism，两线之差 = 开销）

读图：横轴请求率、纵轴平均延迟；同负载下越低越好，两条线的差距就是额外开销。
结果：论文正文报告 32 req/s 时，相比静态分区 TTFT +3 ms、TPOT +4 ms。
结论：额外开销可测但不是零；要与前面动态共享带来的资源收益一起看。

用不利于动态共享的场景，单独测量弹性显存管理的代价

实验评估-生产部署

验证 / 方法：两家公司用 shadow replay，把同一请求流镜像到有 / 无 Prism 的相同集群。

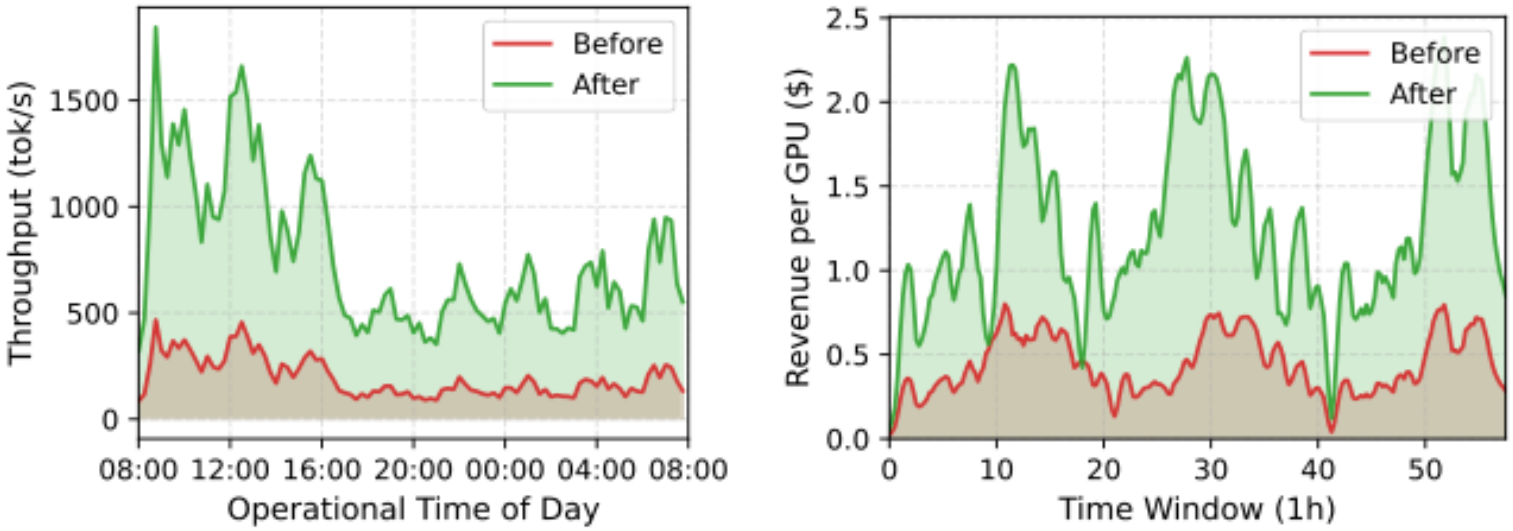


Figure 11：两家公司的生产结果（红 Before = 不用 Prism，绿 After = 用 Prism）

读图：横轴为时间；同一时段，吞吐或每卡收入越高越好，不比较“靠左 / 靠右”。

论文报告：Company A 每卡吞吐平均提升 3.89×；Company B 每卡收入提升 2.86×。

生产结果说明落地收益；

实验评估-超参数敏感性

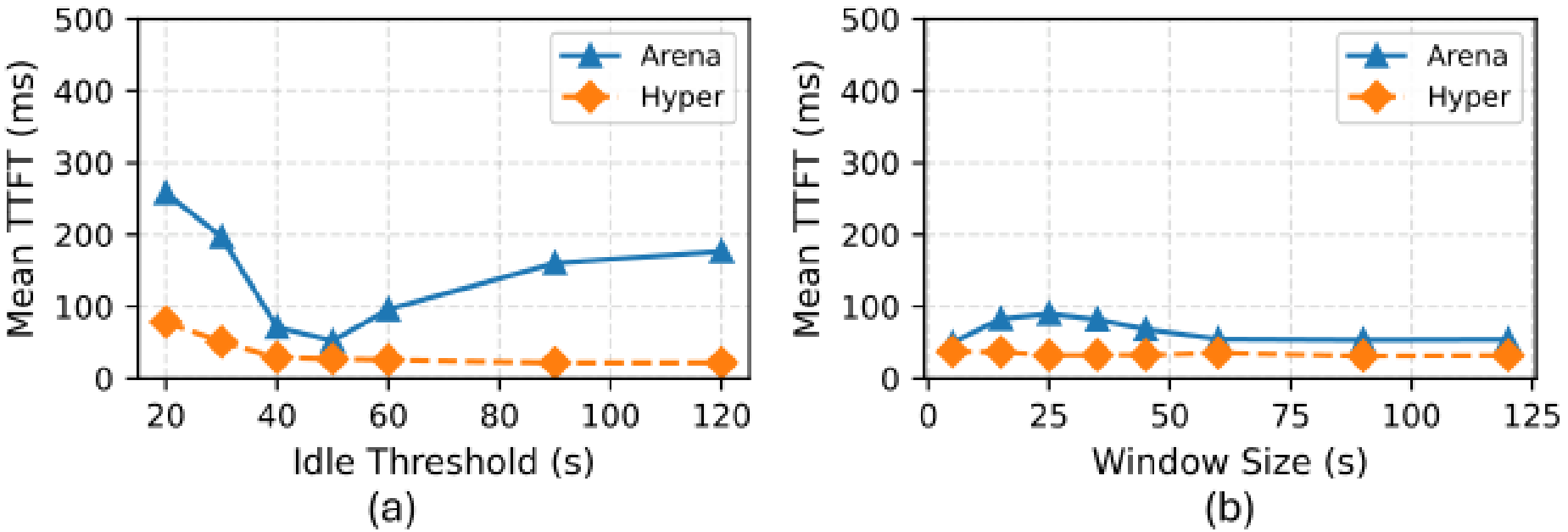


Figure 15：超参数敏感性（纵轴平均 TTFT，越低越好；蓝 Arena，橙 Hyperbolic）

验证 / 方法：在两份 trace 上分别扫描空闲驱逐阈值、KVPR 负载监控窗口，测平均 TTFT。

读图：纵轴延迟越低越好；横轴是参数，不是统一地越左越好；两份 trace 的最优区间可能不同。

结果：论文采用约 45 s 的驱逐阈值、60 s 的监控窗口；Arena 的驱逐曲线呈明显折中。

驱逐阈值需平衡激活代价与显存回收，最优区间随负载变化

总结

问题：模型成组突发、请求剧烈波动 → 用**显存气球化**统一空间共享和时间共享

数据面 kvcached：CUDA 虚拟内存，显存以 2MB 粒度在模型之间、权重与 KV 之间流动

控制面：KVPR 互补放置 + Moore-Hodgson 松弛感知仲裁

引擎池 + 并行加载，模型激活压到 **0.2–1.5s**

结果：TTFT 达标率最高 **3.3×**，TPOT **2×**

同达标率下成本降低 **2× 以上**，或多支撑 **3.5×** 请求

开源：github.com/ovg-project/kvcached，部署在 **1 万张以上** GPU

后续思考

不足

- **只看显存、忽略算力争用**：同卡模型的 prefill / decode 也在抢 SM 与显存带宽

  Hyperbolic 上 TTFT 多组实验停在 0.92–0.97；推测同卡算力干扰是原因之一（论文未分析）

- **策略依赖经验参数，且与负载相关**：KVPR 是启发式，τ 未给取值，阈值与窗口全局统一

  Figure 15 两份 trace 的最优点不同；

延伸思考

- **问题一：能否按模型自适应设驱逐阈值?**

  借鉴 serverless keep-alive，按空闲间隔分布与激活代价定：大模型多留，小模型早让位

- **问题二：算力能否也“气球化”？**

  KVPR 扩展为显存 + 算力压力，避免两个计算密集的模型放在一起

- **问题三：同基座模型能否共享权重页?**

  kvcached 已在页级管理权重，只为差异部分分配显存（参考 Punica）→ 同卡常驻更多模型



Thanks

汇报人：江建谕

2026.11.9